

EXP-1

Student Name: ANSHAJ CHAUHAN

UID: 24BAI70503

Branch: AIT-CSE (AIML)

Section/Group: 24AIT_KRG-1A

Semester: 5

Subject Code: 24CSP-310

Subject Name : Advanced Database Management System

- A. Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The classification categories are:
- "Low Salary": All monthly salaries strictly less than \$20,000.
 - "Average Salary": All monthly salaries in the inclusive range [\$20,000, \$50,000].
 - "High Salary": All monthly salaries strictly greater than \$50,000. The final result table must report all three categories, even if a category contains zero accounts.

SOFTWARE REQUIREMENTS:

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool:

Oracle SQL Developer (for Oracle XE)

CODE:

Query Query History

```
1 CREATE TABLE Accounts (  
2     account_id INT PRIMARY KEY,  
3     income INT NOT NULL  
4 );  
5  
6 INSERT INTO Accounts (account_id, income) VALUES  
7 (3, 108939),  
8 (2, 12747),  
9 (8, 87709),  
10 (6, 91738), |  
11 (7, 45169);  
12
```

```
14 SELECT 'Low Salary' AS category, COUNT(account_id) AS accounts_count  
15 FROM Accounts  
16 WHERE income < 20000  
17  
18 UNION ALL  
19  
20 SELECT 'Average Salary' AS category, COUNT(account_id) AS accounts_count  
21 FROM Accounts  
22 WHERE income >= 20000 AND income <= 50000  
23  
24 UNION ALL  
25  
26 SELECT 'High Salary' AS category, COUNT(account_id) AS accounts_count  
27 FROM Accounts  
28 WHERE income > 50000;
```

	category text	accounts_count bigint
1	Low Salary	1
2	Average Salary	1
3	High Salary	3

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Understand how to use UNION to combine data.
- Learn how to use the AS function .
- Applied count aggregate function.

- B. Design an Employee Management System database architecture by implementing the following operations sequentially:
- Create and populate structural tracking tables for Departments, Employees, and Salaries.
 - Query the system using multi-table Joins to pull clear department breakdowns, apply a 10% salary enhancement across all members of the HR department, and delete any individual salary record dropping strictly below \$30,000.
 - List all remaining active employees whose personal salary exceeds the overall company-wide average salary.

SOFTWARE REQUIREMENTS:

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool:

Oracle SQL Developer (for Oracle XE)

pgAdmin (for PostgreSQL)

CODE :

```
CREATE TABLE Departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL
);

CREATE TABLE Employees (
    emp_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    dept_id INT REFERENCES Departments(dept_id)
);

CREATE TABLE Salaries (
    emp_id INT PRIMARY KEY REFERENCES Employees(emp_id) ON DELETE CASCADE,
    salary INT NOT NULL
);

INSERT INTO Departments (dept_id, dept_name) VALUES
(10, 'HR'), (20, 'IT'), (30, 'Sales');

INSERT INTO Employees (emp_id, name, dept_id) VALUES
(1, 'Alice', 10), (2, 'Bob', 10), (3, 'Charlie', 20), (4, 'David', 20), (5, 'Emma', 30);

INSERT INTO Salaries (emp_id, salary) VALUES
(1, 50000), (2, 25000), (3, 80000), (4, 40000), (5, 45000);
```

```
29 SELECT e.name, d.dept_name, s.salary
30 FROM Employees AS e
31 JOIN Departments AS d
32 ON e.dept_id = d.dept_id
33 JOIN Salaries AS s
34 ON e.emp_id = s.emp_id;
35 |
```

Data Output Messages Notifications

SQL

	name character varying (50)	dept_name character varying (50)	salary integer
1	Alice	HR	50000
2	Bob	HR	25000
3	Charlie	IT	80000
4	David	IT	40000
5	Emma	Sales	45000

```

39  UPDATE Salaries
40  SET salary = salary * 1.10
41  WHERE emp_id IN (
42      SELECT e.emp_id
43      FROM Employees e
44      JOIN Departments d
45      ON
46      e.dept_id = d.dept_id
47      WHERE d.dept_name = 'HR'
48  );
49

```

Data Output Messages Notifications

UPDATE 2

Query returned successfully in 89 msec.

```

50  SELECT e.name, s.salary
51  FROM Employees AS e
52  JOIN Salaries AS s
53  ON e.emp_id = s.emp_id
54  WHERE s.salary > (SELECT AVG(salary) FROM Salaries);

```

Data Output Messages Notifications



	name character varying (50) 🔒	salary integer 🔒
1	Charlie	80000
2	Alice	55000

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Understand how to use JOIN to combine data from multiple tables.
- Learn how to update a table.
- Applied avg function.

C) A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called pizza_toppings. Every customer must choose exactly 3 different toppings to prepare a pizza.

Write a PostgreSQL query to generate every possible 3-topping pizza combination along with its total ingredient cost.

Requirements

Every pizza must contain exactly 3 different toppings. The same topping cannot appear more than once in a pizza. Different arrangements of the same toppings should be considered the same pizza.

Example:

Chicken, Pepperoni, Sausage

Pepperoni, Chicken, Sausage

Sausage, Chicken, Pepperoni

These are all the same pizza, so only one should appear. Display the topping names separated by commas. Display the total ingredient cost of each pizza.

Sort the output:

First by highest total cost

If two pizzas have the same cost, sort them alphabetically.

SOFTWARE REQUIREMENTS:

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool:

Oracle SQL Developer (for Oracle XE)

pgAdmin (for PostgreSQL)

CODE SCREENSHOTS:

```

1 DROP SCHEMA public CASCADE;
2 CREATE SCHEMA public;
3
4 CREATE TABLE pizza_toppings (
5     topping_name VARCHAR(50) PRIMARY KEY,
6     ingredient_cost NUMERIC(4,2) NOT NULL
7 );
8
9
10 INSERT INTO pizza_toppings (topping_name, ingredient_cost) VALUES
11 ('Pepperoni', 0.50),
12 ('Sausage', 0.70),
13 ('Chicken', 0.55),
14 ('Onions', 0.25),
15 ('Extra Cheese', 0.40);

```

```

18 SELECT
19     t1.topping_name || ', ' || t2.topping_name || ', ' || t3.topping_name AS pizza,
20     (t1.ingredient_cost + t2.ingredient_cost + t3.ingredient_cost) AS total_cost
21 FROM pizza_toppings t1
22 JOIN pizza_toppings t2
23     ON t1.topping_name < t2.topping_name
24 JOIN pizza_toppings t3
25     ON t2.topping_name < t3.topping_name
26 ORDER BY total_cost DESC, pizza ASC;

```

Data Output Messages Notifications

	pizza text	total_cost numeric
1	Chicken, Pepperoni, Sausage	1.75
2	Chicken, Extra Cheese, Sausage	1.65
3	Extra Cheese, Pepperoni, Saus...	1.60
4	Chicken, Onions, Sausage	1.50
5	Chicken, Extra Cheese, Pepper...	1.45
6	Onions, Pepperoni, Sausage	1.45
7	Extra Cheese, Onions, Sausage	1.35
8	Chicken, Onions, Pepperoni	1.30
9	Chicken, Extra Cheese, Onions	1.20
10	Extra Cheese, Onions, Pepperoni	1.15

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Understand how to retrieve data from multiple tables using JOIN operations.
- Applied Order by function to order result.
- Used as to change display name.

D) Amazon wants to identify returning customers who made another purchase shortly after their first purchase.

Write a PostgreSQL query to find all users who made another purchase within 1 to 7 days of an earlier purchase.

Requirements

- i) Compare purchases made by the same user only.
- ii) Ignore comparisons where the same transaction is matched with itself.
- iii) The second purchase must occur after the first purchase.
- iv) Same-day purchases must not be considered.
- v) The second purchase must occur within 7 days of the first purchase.
- vi) Display only the unique User IDs that satisfy the above conditions.
- vii) Sort the output in ascending order of user_id.

SOFTWARE REQUIREMENTS:

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool: Oracle SQL Developer (for Oracle XE) pgAdmin (for PostgreSQL)

CODE:

```
CREATE TABLE amazon_transactions (  
    id INT PRIMARY KEY,  
    user_id INT NOT NULL,  
    item VARCHAR(50),  
    created_at TIMESTAMP NOT NULL,  
    revenue INT  
);  
  
INSERT INTO amazon_transactions (id, user_id, item, created_at, revenue) VALUES  
(1, 101, 'biscuit', '2026-03-01 10:00:00', 12),  
(2, 101, 'milk', '2026-03-01 14:00:00', 5),  
(3, 101, 'bread', '2026-03-05 09:00:00', 8),  
(4, 102, 'banana', '2026-03-10 12:00:00', 4),  
(5, 102, 'apple', '2026-03-25 11:00:00', 6),  
(6, 103, 'milk', '2026-03-15 08:00:00', 5),  
(7, 103, 'bread', '2026-03-16 08:00:00', 7);
```

```
21 SELECT DISTINCT t1.user_id
22 FROM amazon_transactions t1
23 JOIN amazon_transactions t2
24 ON t1.user_id = t2.user_id
25 WHERE t1.id <> t2.id
26     AND t2.created_at > t1.created_at
27     AND DATE(t2.created_at) > DATE(t1.created_at)
28     AND t2.created_at <= t1.created_at + INTERVAL '7 days'
29 ORDER BY t1.user_id;
```

Data Output Messages Notifications



	user_id integer
1	101
2	103

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Understand how to retrieve data from multiple tables using JOIN operations.
- understood how to use date .
- Used comparison operator.